LP-III Machine Learning Practicals: Short Notes, Code and Viva Answers

Same format as the DAA and Blockchain notes: for each practical write Title, Aim, Short Theory, Steps, Code, Complexity, Observation and Conclusion. Code is Python for Jupyter Notebook or Google Colab. Put the downloaded CSV file in the same folder as the notebook (or upload it in Colab) so the file names below work.

Practical 1: Uber Fare Prediction (Linear Regression and Random Forest)

Aim: Predict the Uber ride fare. Pre-process the data, identify outliers, check correlation, implement linear regression and random forest regression, and compare R2 and RMSE.

Theory:

- **Data preprocessing:** cleaning raw data (missing values, wrong values, categorical encoding, scaling) so a model can use it.
- **Outliers:** data points far from the rest (for example a fare of 0 or 500 dollars, or a latitude outside the valid range). They are found with a **boxplot** (points beyond 1.5 x IQR from the quartiles) and removed or capped.
- **Haversine formula:** the distance between two points on a sphere from their latitude and longitude. We use it to create a `distance_km` feature from pickup and drop-off coordinates.
- **Correlation:** a value from -1 to +1 showing how strongly two variables move together. A heatmap of the correlation matrix shows which features relate to fare.
- **Linear regression:** fits a straight line (y = b0 + b1x1 + ... + bnxn) to predict a continuous value.
- **Random forest regression:** an ensemble of many decision trees built on random subsets of data; the final prediction is the **average** of all trees.
- **Metrics:** R2 = share of variance explained (closer to 1 is better). MSE = average of squared errors. RMSE = square root of MSE, in the same unit as fare (lower is better).

Steps: load data, drop unused columns, remove nulls, convert datetime and extract hour/day/month, compute distance with Haversine, remove invalid rows and outliers, plot boxplots, draw correlation heatmap, split into train and test, train both models, compare scores.

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

# 1. Load and pre-process
df = pd.read_csv("uber.csv")
df = df.drop(["Unnamed: 0", "key"], axis=1)
df = df.dropna()
df["pickup_datetime"] = pd.to_datetime(df["pickup_datetime"])
df["hour"] = df["pickup_datetime"].dt.hour
df["day"] = df["pickup_datetime"].dt.dayofweek
df["month"] = df["pickup_datetime"].dt.month

# Haversine distance in km
def haversine(lat1, lon1, lat2, lon2):
    R = 6371
    lat1, lon1, lat2, lon2 = map(np.radians, [lat1, lon1, lat2, lon2])
    a = np.sin((lat2 - lat1) / 2) ** 2 + np.cos(lat1) * np.cos(lat2) *
np.sin((lon2 - lon1) / 2) ** 2
    return 2 * R * np.arcsin(np.sqrt(a))

df["distance_km"] = haversine(df["pickup_latitude"], df["pickup_longitude"],
                              df["dropoff_latitude"],
df["dropoff_longitude"])
df = df.drop("pickup_datetime", axis=1)

# 2. Remove invalid rows
df = df[(df["fare_amount"] > 0) & (df["passenger_count"] > 0) &
(df["distance_km"] > 0)]
df = df[df["pickup_latitude"].between(-90, 90) &
df["dropoff_latitude"].between(-90, 90)]
df = df[df["pickup_longitude"].between(-180, 180) &
df["dropoff_longitude"].between(-180, 180)]

# 3. Identify outliers (boxplot) and remove them with the IQR rule
df[["fare_amount", "distance_km"]].plot(kind="box", subplots=True, figsize=
```

```
(8, 4))
plt.show()

def remove_outliers(data, col):
    q1, q3 = data[col].quantile(0.25), data[col].quantile(0.75)
    iqr = q3 - q1
    return data[(data[col] >= q1 - 1.5 * iqr) & (data[col] <= q3 + 1.5 *
iqr)]

df = remove_outliers(df, "fare_amount")
df = remove_outliers(df, "distance_km")

# 4. Correlation
plt.figure(figsize=(9, 6))
sns.heatmap(df.corr(), annot=True, cmap="coolwarm")
plt.show()

# 5. Split
X = df.drop("fare_amount", axis=1)
y = df["fare_amount"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# 6. Models
lr = LinearRegression().fit(X_train, y_train)
rf = RandomForestRegressor(n_estimators=100, random_state=42,
n_jobs=-1).fit(X_train, y_train)

# 7. Evaluate
for name, model in [("Linear Regression", lr), ("Random Forest", rf)]:
    pred = model.predict(X_test)
    print(name)
    print("  R2   :", r2_score(y_test, pred))
    print("  RMSE :", np.sqrt(mean_squared_error(y_test, pred)))
    print("  MAE  :", mean_absolute_error(y_test, pred))
```

Complexity: Linear regression training is about O(n x d^2 + d^3) (n rows, d features); prediction is O(d). Random forest training is about O(T x n log n x d) for T trees, and prediction is O(T x depth).

Observation: Random forest usually gives a higher R2 and a lower RMSE than linear regression, because fare depends non-linearly on distance, time of day and location. Write down your own R2 and RMSE values for both models.

Conclusion: Correlation was studied and linear regression and random forest regression models were implemented and compared.

Viva questions and answers

1. What is data preprocessing? Preparing raw data so a machine learning model can use it: getting the data, handling missing values, encoding categorical data, splitting into train and test sets, and feature scaling. Real data contains noise and missing values, and cleaning it improves accuracy and efficiency.

2. Define outliers. Data points that lie far outside the expected range of the rest of the data, often caused by errors or rare events. Types: global (point) outliers, collective outliers (a group that deviates together) and contextual outliers (unusual only in a certain context, such as 45 degrees Celsius in the rainy season).

3. What is linear regression? A supervised algorithm that predicts a continuous value (price, salary, sales) by fitting a straight line that shows the relationship between the dependent variable and one or more independent variables.

4. What is the random forest algorithm? An ensemble method that builds many decision trees on random subsets of the data and combines them: it takes the average of the trees for regression and the majority vote for classification. More trees improve accuracy and reduce overfitting.

5. Explain pandas and numpy. NumPy provides fast n-dimensional arrays and mathematical functions. **Pandas** is built on NumPy and provides the DataFrame, used to load, clean, filter and analyze tabular data (for example `read_csv`, `dropna`, `groupby`).

Practical 2: Email Spam Classification (KNN and SVM)

Aim: Classify emails as spam or not spam using K-Nearest Neighbors and Support Vector Machine, and analyze their performance.

Theory:

- **Binary classification:** a task with two classes. Here "not spam" is the normal state (label 0) and "spam" is the abnormal state (label 1).
- **KNN:** a lazy learner that stores the training data and classifies a new email by the majority class among its K nearest neighbours (usually by Euclidean distance).
- **SVM:** finds the best separating **hyperplane** that has the maximum margin between the two classes. The points closest to the boundary are the **support vectors**. A linear SVM handles linearly separable data; kernels (rbf, polynomial) handle non-linear data.

- **Train-test split:** the data is split into a training set (about 75 to 80 percent) and a test set (the rest) to check performance on unseen data.

Dataset: `emails.csv` (5172 emails, 3000 word-count columns, last column `Prediction`: 1 = spam, 0 = not spam).

Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

df = pd.read_csv("emails.csv")
print(df.isnull().sum().sum())                # check missing values
df = df.drop("Email No.", axis=1)             # remove the id column

X = df.drop("Prediction", axis=1)
y = df["Prediction"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# KNN
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
knn_pred = knn.predict(X_test)

# SVM
svm = SVC(kernel="linear", C=1.0)
svm.fit(X_train, y_train)
svm_pred = svm.predict(X_test)

for name, pred in [("KNN", knn_pred), ("SVM", svm_pred)]:
```

```
    print("=====", name, "=====")
    print("Accuracy:", accuracy_score(y_test, pred))
    print("Confusion matrix:\n", confusion_matrix(y_test, pred))
    print(classification_report(y_test, pred))
```

Complexity: KNN training is O(1) (it only stores data), but each prediction costs O(n x d), so it is slow on large data. SVM training is roughly O(n^2) to O(n^3) and prediction is O(support vectors x d).

Observation: compare accuracy, precision and recall of the two models. SVM usually performs better on this high-dimensional text data, while KNN is slower at prediction time.

Conclusion: Email spam detection was implemented using KNN and SVM and their performance was compared.

Viva questions and answers

1. What is binary classification? Classification with only two class labels, for example spam or not spam, churn or no churn. One class is the normal state (label 0) and the other is the abnormal state (label 1). Typical algorithms: logistic regression, KNN, decision trees, SVM and naive Bayes.

2. Explain Support Vector Machine. A supervised algorithm for classification and regression. It finds the hyperplane that separates the classes with the maximum margin; the extreme points that define the margin are called support vectors. Linear SVM is for linearly separable data, and non-linear SVM uses kernels. Uses: face detection, image classification, text categorization.

3. Explain the K-Nearest Neighbour algorithm. A simple supervised, non-parametric, lazy-learning algorithm. It stores all training data; to classify a new point it finds the K closest points by distance and assigns the majority class among them (or the average value for regression). The value of K is chosen by testing different values.

Practical 3: Bank Customer Churn (Neural Network Classifier)

Aim: Build a neural network classifier that predicts whether a bank customer will leave in the next 6 months. Normalize the data, build and improve the model, and print the accuracy and confusion matrix.

Theory:

- **Artificial Neural Network (ANN):** layers of connected neurons modelled on the brain. The **input layer** takes the features, **hidden layers** compute patterns, and the **output layer** gives the result. Each neuron computes a weighted sum of inputs plus a bias and passes it through an **activation function** (ReLU in hidden layers, sigmoid at the output for binary classification).
- **Keras / TensorFlow:** TensorFlow is Google's deep learning library; Keras is its high-level API for building networks quickly.
- **Normalization:** rescaling features to a common range. Min-Max scaling: Xn = (X - Xmin) / (Xmax - Xmin) gives values from 0 to 1. Standardization (Z-score): (X - mean) / std. Needed because CreditScore, Balance and Age have very different ranges.
- **Confusion matrix:** table of TP, TN, FP and FN used to compute accuracy.

Dataset: `Churn_Modelling.csv` (10,000 rows, 14 columns; target column `Exited`: 1 = customer left).

Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping

# 1. Read the data
df = pd.read_csv("Churn_Modelling.csv")

# 2. Features and target (drop RowNumber, CustomerId, Surname)
X = df.drop(["RowNumber", "CustomerId", "Surname", "Exited"], axis=1)
y = df["Exited"]
X = pd.get_dummies(X, columns=["Geography", "Gender"], drop_first=True)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# 3. Normalize (fit on train only)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 4. Build the model
```

```
model = Sequential([
    Input(shape=(X_train.shape[1],)),
    Dense(16, activation="relu"),
    Dropout(0.2),                        # improvement: reduce overfitting
    Dense(8, activation="relu"),
    Dense(1, activation="sigmoid"),
])
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=
["accuracy"])

early = EarlyStopping(monitor="val_loss", patience=5,
restore_best_weights=True)
model.fit(X_train, y_train, epochs=100, batch_size=32,
          validation_split=0.1, callbacks=[early], verbose=1)

# 5. Evaluate
y_pred = (model.predict(X_test) > 0.5).astype(int)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion matrix:\n", confusion_matrix(y_test, y_pred))
```

Points of improvement (step 4 of the assignment): add more hidden layers or neurons, add dropout, use early stopping, tune the learning rate and batch size, handle class imbalance (only about 20 percent of customers leave) with `class_weight`, and tune the 0.5 threshold. Report the accuracy before and after improvement.

Complexity: training cost is about O(epochs x n x number of weights); prediction is O(number of weights) per sample.

Conclusion: A neural network classifier was built to predict customer churn, and its accuracy and confusion matrix were obtained.

Viva questions and answers

1. What is an artificial neural network? A computing model inspired by the human brain, made of interconnected neurons arranged in input, hidden and output layers. Inputs are multiplied by weights, summed with a bias and passed through an activation function. The network learns by adjusting weights using backpropagation.

2. What is an activation function? Name some. A function that decides whether a neuron fires and adds non-linearity. Examples: ReLU, sigmoid, tanh and softmax.

3. What are Keras and TensorFlow? TensorFlow is an open-source deep learning framework by Google (released 2015) that runs on CPUs and GPUs. Keras is a user-friendly high-level API (now part of TensorFlow) for building and training neural networks.

4. What is normalization? What is the difference from standardization? Normalization (Min-Max scaling) rescales values to the range 0 to 1. Standardization subtracts the mean and divides by the standard deviation, so data has mean 0 and standard deviation 1 with no fixed range. Scaling is needed so that features with large ranges do not dominate the model.

5. What is a confusion matrix? A table that compares actual and predicted classes: TP, TN, FP (Type I error) and FN (Type II error). From it we compute accuracy, error rate, precision and recall.

6. Why fit the scaler only on the training data? To avoid data leakage: the test set must stay unseen, so its statistics should not influence the scaling.

Practical 4: KNN on Diabetes Dataset

Aim: Implement K-Nearest Neighbors on `diabetes.csv` and compute the confusion matrix, accuracy, error rate, precision and recall.

Theory:

- **KNN:** a supervised, non-parametric, lazy-learner algorithm. It assumes similar things lie close together. A new point is assigned the class held by the majority of its K nearest neighbours, using distance such as Euclidean: d = sqrt(sum of (xi - yi)^2).
- **Confusion matrix** (2 x 2):

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

- Accuracy = (TP + TN) / total. Error rate = (FP + FN) / total = 1 - accuracy. Precision = TP / (TP + FP). Recall = TP / (TP + FN).

Dataset: `diabetes.csv` (768 patients, 8 diagnostic features such as Glucose, BMI, Age, target `Outcome`: 1 = diabetic).

Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, accuracy_score,
precision_score, recall_score

df = pd.read_csv("diabetes.csv")

X = df.drop("Outcome", axis=1)
y = df["Outcome"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

scaler = StandardScaler()          # KNN uses distance, so scaling is
important
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)

cm = confusion_matrix(y_test, y_pred)
tn, fp, fn, tp = cm.ravel()
print("Confusion matrix:\n", cm)
print("Accuracy   :", accuracy_score(y_test, y_pred))
print("Error rate :", 1 - accuracy_score(y_test, y_pred))
print("Precision  :", precision_score(y_test, y_pred))
print("Recall     :", recall_score(y_test, y_pred))

# Optional: find the best K
for k in range(1, 16, 2):
    m = KNeighborsClassifier(n_neighbors=k).fit(X_train, y_train)
    print("K =", k, "accuracy =", m.score(X_test, y_test))
```

Complexity: training O(1) (stores data); prediction O(n x d) per test sample, so KNN becomes slow on big datasets. Space is O(n x d).

Observation: write the confusion matrix values (TP, TN, FP, FN) and the four metrics. For a medical problem, **recall** is important because a missed diabetic patient (FN) is costly.

Conclusion: KNN was implemented on the diabetes dataset and the confusion matrix, accuracy, error rate, precision and recall were computed.

Viva questions and answers

1. Explain the KNN algorithm. Choose K, compute the distance from the new point to all training points, pick the K nearest, and assign the majority class. It is a lazy learner because it does no training; all work happens at prediction time.

2. How do you choose K? Try several values (usually odd, to avoid ties) and pick the one with the best validation accuracy. A very small K overfits (noisy), a very large K underfits.

3. Why scale the data for KNN? Because KNN uses distance, so a feature with large values (like Insulin) would dominate small ones (like Pregnancies).

4. What is the difference between precision and recall? Precision is the fraction of predicted positives that are really positive (TP / (TP + FP)). Recall is the fraction of actual positives that were found (TP / (TP + FN)).

Practical 5: K-Means and Hierarchical Clustering on Sales Data (Elbow Method)

Aim: Implement K-Means (and hierarchical) clustering on `sales_data_sample.csv` and find the number of clusters using the elbow method.

Theory:

- **K-Means:** an unsupervised, centroid-based algorithm. Steps: (1) choose K, (2) pick K random centroids, (3) assign each point to the nearest centroid, (4) recompute each centroid as the mean of its cluster, (5) repeat steps 3 and 4 until assignments stop changing.
- **Elbow method:** run K-Means for K = 1 to 10 and compute **WCSS** (within-cluster sum of squares, the sum of squared distances of points to their centroid). Plot WCSS against K; the point where the curve bends like an elbow is the best K.
- **Hierarchical clustering:** builds a tree of clusters (dendrogram) without fixing K first. **Agglomerative** is bottom-up (each point starts as a cluster and the closest pairs are merged); **divisive** is top-down. Linkage methods: single, complete, average, centroid.
- **PCA:** a dimensionality-reduction method that converts correlated features into uncorrelated principal components. We use it to reduce the data to 2 dimensions so the clusters can be plotted.

Code:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.decomposition import PCA
import scipy.cluster.hierarchy as sch

df = pd.read_csv("sales_data_sample.csv", encoding="latin1")
X = df[["QUANTITYORDERED", "PRICEEACH", "SALES", "MSRP"]].dropna()
X_scaled = StandardScaler().fit_transform(X)

# Elbow method
wcss = []
for k in range(1, 11):
    km = KMeans(n_clusters=k, init="k-means++", n_init=10, random_state=42)
    km.fit(X_scaled)
    wcss.append(km.inertia_)

plt.plot(range(1, 11), wcss, marker="o")
plt.title("Elbow Method")
plt.xlabel("Number of clusters (K)")
plt.ylabel("WCSS")
plt.show()

# K-Means with the K read from the elbow (3 is a typical choice here; use
your own plot)
K = 3
kmeans = KMeans(n_clusters=K, init="k-means++", n_init=10, random_state=42)
labels = kmeans.fit_predict(X_scaled)

# Visualize using PCA
pca = PCA(n_components=2)
comp = pca.fit_transform(X_scaled)
plt.scatter(comp[:, 0], comp[:, 1], c=labels, cmap="viridis", s=15)
plt.title("K-Means clusters (PCA view)")
plt.show()

# Hierarchical clustering
sch.dendrogram(sch.linkage(X_scaled, method="ward"))
plt.title("Dendrogram")
```

```
plt.show()

hc = AgglomerativeClustering(n_clusters=K, linkage="ward")
hc_labels = hc.fit_predict(X_scaled)
```

Complexity: K-Means is O(n x K x d x i) for i iterations, which is fast for large data. Hierarchical (agglomerative) clustering is O(n^2 log n) to O(n^3) time and O(n^2) space, so it suits only small or medium data.

Observation: note the K at the elbow of the WCSS graph, the cluster sizes, and what each cluster represents (for example low, medium and high value orders).

Conclusion: K-Means and hierarchical clustering were implemented on the sales data and the number of clusters was found with the elbow method.

Viva questions and answers

1. Explain K-Means clustering. An unsupervised algorithm that divides unlabeled data into K clusters. It repeatedly assigns each point to the nearest centroid and moves each centroid to the mean of its cluster, until no point changes cluster. It minimizes the sum of squared distances inside clusters.

2. Explain hierarchical clustering. An unsupervised method that builds a hierarchy of clusters shown as a dendrogram, so K need not be fixed in advance. Agglomerative (bottom-up) merges the two closest clusters at each step; divisive (top-down) splits clusters. Distance between clusters is set by a linkage method (single, complete, average, centroid).

3. Explain Principal Component Analysis. A dimensionality-reduction technique that transforms correlated features into a smaller set of uncorrelated features called principal components, which keep the maximum variance. Steps: standardize the data, compute the covariance matrix, find eigenvalues and eigenvectors, sort them in decreasing order, and project the data onto the top components.

4. Explain the elbow method. A way to choose the number of clusters. Run K-Means for K = 1 to 10, compute WCSS for each, and plot WCSS against K. WCSS falls as K increases; the point where the fall suddenly slows, forming an elbow, is taken as the best K.

5. Define eigenvectors. For a square matrix A, a non-zero vector v is an eigenvector if Av = λv, that is, multiplying by A only scales v by a number λ (the eigenvalue). In PCA the eigenvectors of the covariance matrix give the directions of the principal components.

Quick Comparison of Algorithms Used

Practical	Algorithm	Type	Output	Main metric
1. Uber fare	Linear Regression, Random Forest	Supervised, regression	Fare (number)	R2, RMSE
2. Spam	KNN, SVM	Supervised, classification	Spam or not	Accuracy, confusion matrix
3. Churn	Neural network	Supervised, classification	Leave or stay	Accuracy, confusion matrix
4. Diabetes	KNN	Supervised, classification	Diabetic or not	Accuracy, precision, recall
5. Sales	K-Means, Hierarchical	Unsupervised, clustering	Cluster label	WCSS (elbow)